

EcoSphere

Real-Time Voice AI Sales Agent

Hackathon Project Document

Team: Winners4 | Round: II

Team Members

Name	Role	Email
Sumodip Patra	Agent Brain (LangGraph)	sumodeeppatra@gmail.com
Harsh Kalbande	Voice Pipeline Engineer	harshkalbande.10@gmail.com
Kanishk Patil	Backend & Integration	kanishkpatil.kp21@gmail.com
Yash Zunzurkar	Frontend, Demo Flow Lead	yashzunzurkar@gmail.com

1. Problem Statement

Build a real-time voice AI sales agent that can conduct a complete customer qualification and sales conversation.

Problem Description

Sales calls are scripted and brittle - bots break when customers interrupt, change requirements, or ask off-script questions. This creates poor customer experience. A real-time voice agent that understands, adapts, and remembers context throughout a live conversation can drive meaningful sales outcomes.

Proposed Solution

We propose a real-time voice AI sales agent that conducts natural conversations instead of following fixed scripts. With Agora Conversational AI, we harness the AI agent to handle interruptions, change requirements, or maintain context like a real salesperson.

At its core is a Stateful LangGraph agent that maintains session memory across the entire call, and retrieves live product and pricing information through RAG. It is also integrated with a calendar to schedule meetings with human agents when the situation requires human intervention.

What Makes This Unique

- Combining conversational memory, dynamic objection handling, and tool calling in one voice-native pipeline
- Moving every call toward a clear, actionable outcome
- Dynamic LangGraph routing instead of decision trees
- Persistent memories instead of stateless turns
- RAG-grounded answers instead of hallucination-prone free generation

2. System Architecture

High-Level Flow

Customer speaks -> Agora RTC Channel -> Agora ConvoAI Engine -> Your Custom LLM Server (LangGraph + RAG + Tools) -> Response -> TTS -> Customer

Components

- Agora Voice Transport: Low-latency RTC channels connect customer to AI agent globally
- Agora ConvoAI Engine: Orchestrates STT, LLM reasoning, and TTS in one flow
- LangGraph Agent: Stateful multi-turn agent with memory and dynamic routing
- RAG Knowledge Base: Product catalog + pricing info via ChromaDB vector search
- Tool Calling: Calendar scheduling and product lookup
- Frontend: Browser-based test UI for voice calls

Agora Technology Utilization

Feature	How It Works
Real-Time Voice Transport	Low-latency RTC globally
ASR -> LLM -> TTS Pipeline	Agora ConvoAI orchestrates
Interruption Handling	Voice detection stops agent
Noise Suppression	95% background noise filtered
Selective Attention	Locks onto primary speaker
Session Management	Channels, tokens, scheduling

3. Current Status - What Is Built

Existing Files

File	Status	Description
voice_pipeline.py	BUILT	Agora pipeline (Deepgram->OpenAI->MiniMax)
test_agora.py	BUILT	Simple Agora client test
test_call.html	BUILT	Browser UI for voice calls
text_call.html	BUILT	Browser UI with token
.env	BUILT	Credentials (Agora App ID/Cert)

Key Code - voice_pipeline.py

```
from agora_agent import Agent, Agora, Area, DeepgramSTT, OpenAI, MiniMaxTTS

client = Agora(area=Area.US, app_id="66d8b08d...",
               app_certificate="1e3fbbb8...")

agent = (
    Agent(client=client, turn_detection={"language": "en-US"})
    .with_stt(DeepgramSTT(model="nova-3", language="en"))
    .with_llm(OpenAI(model="gpt-4o-mini",
                    system_messages=[{"role": "system", "content": AGENT_PROMPT}],
                    greeting_message=GREETING, max_history=50,
                    params={"max_tokens": 256, "temperature": 0.7}))
    .with_tts(MiniMaxTTS(model="speech_2_6_turbo",
                        voice_id="English_captivating_female1"))
)

session = agent.create_session(
    channel=f"sales-call-{int(time.time())}",
    agent_uid="100001", remote_uids=["*"],
    idle_timeout=60, debug=True)

session.start()
```

4. What Needs To Be Built

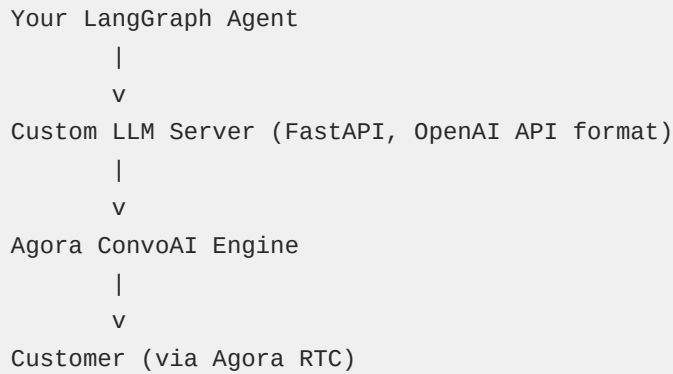
Remaining Components

Component	Priority	Owner	Status
LangGraph Stateful Agent	CRITICAL	Harsh (Voice)	NOT BUILT
RAG Knowledge Base	CRITICAL	Sumodip (Brain)	NOT BUILT
Interruption Handling	CRITICAL	Harsh (Voice)	NOT BUILT
Calendar Integration	HIGH	Kanishk (Backend)	NOT BUILT
Dynamic Objection Handling	HIGH	Sumodip (Brain)	NOT BUILT
Frontend Dashboard	HIGH	Yash (Frontend)	NOT BUILT
Analytics Dashboard	NICE-TO-HAVE	Team	NOT BUILT
Multi-Language Support	NICE-TO-HAVE	Team	NOT BUILT

5. Implementation Plan

Architecture Overview

Agora supports Custom LLM via OpenAI API-compatible endpoint. We wrap our LangGraph agent behind this endpoint:



Step 1: Install Dependencies

```
pip3 install langgraph langchain langchain-openai
pip3 install chromadb fastapi uvicorn
pip3 install python-dotenv
```

Step 2: Define Agent State (agent/state.py)

The state holds conversation memory, customer info, and current intent:

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    customer_info: dict
    current_intent: str
    handoff_needed: bool
```

Step 3: Build LangGraph Graph (agent/graph.py)

The graph is a state machine that routes conversations:

```
from langgraph.graph import StateGraph

graph = StateGraph(AgentState)

# Add nodes
graph.add_node("classify_intent", classify_intent)
graph.add_node("handle_pricing", handle_pricing)
graph.add_node("handle_objection", handle_objection)
graph.add_node("handle_demo", handle_demo)

# Add routing logic
graph.add_conditional_edges(
    "classify_intent",
    route_intent,
    {
        "pricing": "handle_pricing",
        "objection": "handle_objection",
        "demo": "handle_demo",
        "general": "handle_general",
    }
)

graph.set_entry_point("classify_intent")
```

Step 4: Add Tool Calling (tools/)

```
from langchain_core.tools import tool

@tool
def lookup_product(query: str) -> str:
    """Search product catalog for pricing info"""
    # Search ChromaDB, return product info
    results = product_db.similarity_search(query)
    return format_results(results)

@tool
def schedule_meeting(name: str, email: str, context: str) -> str:
    """Schedule a meeting with a human sales agent"""
    # Create calendar event with full call context
    return f"Meeting scheduled for {name}"
```

Step 5: Create FastAPI Server (server/main.py)

This server exposes your LangGraph agent as an OpenAI-compatible endpoint:

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import json

app = FastAPI()

@app.post("/chat/completions")
async def chat_completions(request: ChatRequest):
    # 1. Run LangGraph agent with user message
    result = agent_graph.invoke({
        "messages": request.messages,
        "customer_info": {},
        "current_intent": "",
        "handoff_needed": False
    })

    # 2. Stream response in OpenAI format
    async def generate():
        for chunk in result["response"]:
            yield f"data: {json.dumps(chunk)}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(generate(),
                             media_type="text/event-stream")
```

Step 6: Update Voice Pipeline (voice_pipeline.py)

```
from agora_agent import Agent, Agora, Area, DeepgramSTT, MiniMaxTTS, CustomLLM

agent = (
    Agent(client=client, turn_detection={"language": "en-US"})
    .with_stt(DeepgramSTT(model="nova-3", language="en"))
    .with_llm(CustomLLM(
        api_key="your-key",
        base_url="http://localhost:8000/chat/completions",
        model="gpt-4o-mini",
        system_messages=[{"role": "system", "content": SALES_PROMPT}],
        greeting_message="Hi! I am your AI sales assistant...",
        max_history=50,
    ))
    .with_tts(MiniMaxTTS(model="speech_2_6_turbo",
        voice_id="English_captivating_female1"))
)
```


6. Interruption Handling

How Agora Handles Interruptions

Agora ConvoAI Engine has built-in interruption support. Voice interruption is automatic - when the engine detects user voice input, it automatically stops the agent response.

Configuration

```
agent = Agent(client=client)
.with_turn_detection({
    "mode": "default",
    "config": {
        "speech_threshold": 0.5,
        "start_of_speech": {
            "mode": "vad",
            "vad_config": {
                "interrupt_duration_ms": 160,
                "speaking_interrupt_duration_ms": 320,
                "prefix_padding_ms": 800,
            },
        },
        "end_of_speech": {
            "mode": "semantic",
            "semantic_config": {
                "silence_duration_ms": 320,
                "max_wait_ms": 3000,
            },
        },
    },
})
```

Manual Interruption

```
# Server-side interrupt
session.interrupt()

# Or via REST API
# POST /agents/{agentId}/interrupt
```

Test Scenarios

Scenario	Expected Behavior
Customer talks over pricing	AI stops, addresses new question
Customer says "wait" mid-sentence	AI yields immediately

Customer stays silent	AI finishes response normally
Customer changes topic	AI adapts to new topic

7. RAG Knowledge Base

Product Knowledge Structure

```
# knowledge/products.json
{
  "products": [
    {
      "name": "Enterprise Plan",
      "price": "$99/month",
      "features": ["Unlimited calls", "Analytics", "Priority support"],
      "target": "Large teams"
    },
    {
      "name": "Pro Plan",
      "price": "$49/month",
      "features": ["1000 calls/mo", "Basic analytics"],
      "target": "Growing teams"
    }
  ]
}
```

ChromaDB Setup

```
import chromadb

client = chromadb.Client()
collection = client.create_collection("products")

# Add product data
collection.add(
    documents=["Enterprise Plan at $99/month..."],
    metadatas=[{"type": "pricing"}],
    ids=["prod_1"]
)

# Query
results = collection.query(
    query_texts=["How much does enterprise cost?"],
    n_results=3
)
```

8. Tool Calling

Calendar Tool

```
@tool
def schedule_meeting(customer_name: str, email: str,
                     topic: str, context: str) -> str:
    """Schedule a meeting with human sales agent"""
    # Integrate with Google Calendar API
    event = {
        "summary": f"Sales Follow-up: {customer_name}",
        "description": f"Context: {context}",
        "attendees": [{"email": email}],
    }
    # Create event and return confirmation
    return f"Meeting scheduled for {customer_name} at {email}"
```

Product Lookup Tool

```
@tool
def lookup_product(query: str) -> str:
    """Search product catalog for information"""
    results = product_collection.query(
        query_texts=[query], n_results=3
    )
    return format_product_results(results)
```

9. Extensibility & Future Plans

Beyond the Hackathon

- Multi-language support using Agora multilingual ASR/TTS
- Sentiment-based escalation triggers (not just keywords)
- Analytics dashboard to track call outcomes and conversion rates
- Generalize architecture to customer support, onboarding, appointment scheduling
- Real CRM integration (Salesforce, HubSpot) instead of Google Sheets mock

Validation

We have mapped example scenarios (pricing questions, competitor interrupts, user count changes, enterprise demo requests) to specific nodes in our LangGraph design. We know precisely how each transition will be handled before we start building.

Constraints (Hackathon)

- Google Sheets as mock CRM instead of live enterprise CRM
- Same tool calling pattern extends to real CRM API in production

10. Quick Reference - Key Commands

Install Dependencies

```
pip3 install langgraph langchain langchain-openai
pip3 install chromadb fastapi uvicorn python-dotenv
pip3 install agora-agents
```

Run the Server

```
# Terminal 1: Start the LangGraph server
cd server
python3 main.py
# Server runs at http://localhost:8000

# Terminal 2: Start the voice pipeline
python3 voice_pipeline.py
```

Test the Voice Call

```
# Open test_call.html in browser
# Paste the channel name from terminal
# Click "Join Call"
# Start speaking to the AI agent
```

Project Structure

```
Winners4/
|- agent/
|   |- state.py          # Agent state definition
|   |- graph.py          # LangGraph graph
|- knowledge/
|   |- products.json     # Product catalog
|   |- rag.py            # RAG integration
|- tools/
|   |- calendar.py       # Meeting scheduling
|   |- product_lookup.py # Product search
|- server/
|   |- main.py           # FastAPI server
|- voice_pipeline.py      # Agora voice pipeline
|- test_call.html        # Browser test UI
|- .env                  # Credentials
```